

**UNIVERSIDADE VILA VELHA
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

Ivan e Flavio

OBJETOS OCULTOS

Documentação do desenvolvimento de um jogo autoral de objetos ocultos

VILA VELHA – ES
2026

Ivan e Flavio

OBJETOS OCULTOS

Documentação do desenvolvimento de um jogo autoral de objetos ocultos

Documentação apresentada como requisito avaliativo da atividade sobre UML, GUI, Jogos e Gestão, do Curso de Ciência da Computação da Universidade Vila Velha (UVV), sob orientação do professor Vinícius Rosalem. Em acordo com o professor, optou-se pelo desenvolvimento de um jogo autoral, alternativo ao catálogo inicial da atividade.

VILA VELHA – ES
2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 DESCRIÇÃO DO PROJETO.....	4
2.1 Contexto do Projeto	4
2.2 Tecnologias Utilizadas	4
2.3 Requisitos Atendidos	5
3 PLANEJAMENTO DAS ATIVIDADES	6
4 MODELAGEM UML	7
4.1 Diagrama de Casos de Uso	7
4.2 Diagrama de Classes.....	7
5 DESENVOLVIMENTO DO JOGO.....	8
5.1 A Ideia	8
5.2 As Imagens	8
5.3 Tela de Início	9
5.4 Tela Jogável.....	10
5.5 Imagens e suas Opacidades	12
5.6 Animação e Função de Esmaecer.....	12
5.7 Botão de Dica	13
5.8 Vitória Verificada na Função Esmaecer.....	14
6 CONSIDERAÇÕES FINAIS	20

1 INTRODUÇÃO

O Objetos Ocultos é um jogo desenvolvido inteiramente pela dupla em C#, utilizando Windows Presentation Foundation (WPF), para a atividade avaliativa de UML, GUI, Jogos e Gestão do Curso de Ciência da Computação da Universidade Vila Velha (UVV). A proposta original da atividade previa a escolha de dois jogos já existentes no catálogo “UVV Steam” para posterior contribuição; no nosso caso, entretanto, ficou combinado com o professor Vinícius Rosalem que o desenvolvimento de um jogo autoral, criado do zero, seria igualmente válido para a avaliação.

A mecânica do jogo é direta: o jogador é apresentado a uma sala iluminada por velas e repleta de objetos de cenário, e precisa localizar nove itens escondidos na composição da imagem de fundo. A cada objeto encontrado, ele desaparece suavemente da cena e é marcado na lista exibida na parte inferior da tela; quando o último item é encontrado, uma tela de vitória é exibida.

Este documento descreve, em ordem, a ideia do jogo, os recursos visuais utilizados, a construção das telas, a lógica de opacidade e animação por trás da mecânica de “achar o objeto”, o botão de dica, a verificação de vitória e o planejamento das atividades da dupla.

2 DESCRIÇÃO DO PROJETO

2.1 Contexto do Projeto

Diferentemente do que é sugerido no enunciado da atividade, não escolhemos nenhum dos jogos disponibilizados no catálogo “UVV Steam”. Depois de avaliarmos as opções, propusemos ao professor Vinícius Rosalem o desenvolvimento de um jogo próprio, e ele validou essa alternativa como um formato igualmente aceitável para a atividade. Assim, todo o sistema aqui documentado — telas, lógica, imagens e estrutura — foi criado pela dupla, sem partir de um projeto pré-existente.

2.2 Tecnologias Utilizadas

O jogo foi desenvolvido em C#, sobre o .NET (net10.0-windows), utilizando Windows Presentation Foundation (WPF) para a interface gráfica. Não há dependências externas via NuGet: toda a interface é construída com os controles nativos do WPF (Window, Grid, Image,

Label, Button, Rectangle) e as animações utilizam a biblioteca de animação do próprio framework (System.Windows.Media.Animation). A aplicação inicia pela MainWindow, conforme definido em App.xaml (StartupUri="MainWindow.xaml").

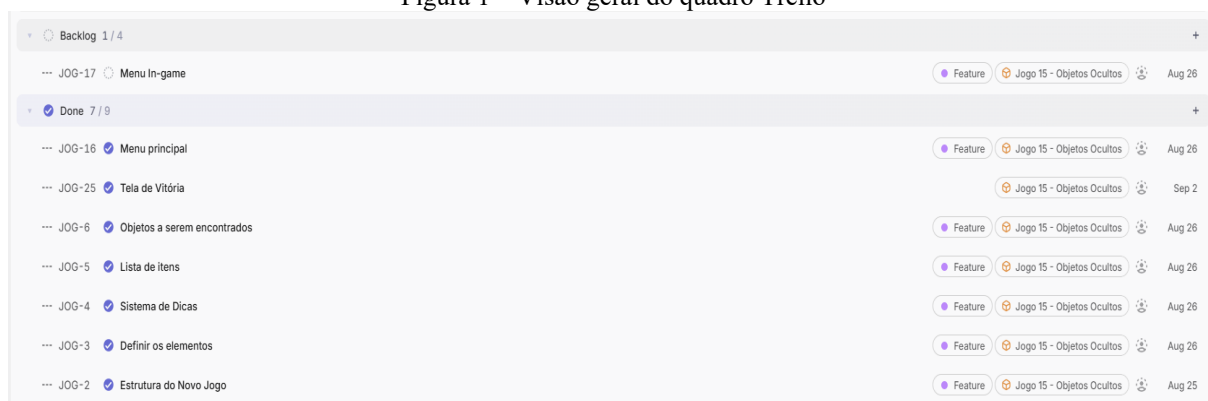
2.3 Requisitos Atendidos

Em relação aos requisitos obrigatórios da atividade, este documento apresenta: (i) a descrição completa do sistema desenvolvido, já que se trata de um jogo autoral e não de uma contribuição a um projeto de terceiros; (ii) o detalhamento de cada parte construída, na forma de adições sucessivas ao sistema; (iii) o diagrama de casos de uso elaborado a partir da lógica implementada; e (iv) o planejamento das atividades individuais por meio de ferramenta de gestão de tarefas.

3 PLANEJAMENTO DAS ATIVIDADES

O planejamento e o acompanhamento das tarefas individuais da dupla foram feitos por meio do Trello, organizado nas colunas Backlog, To Do, Doing e Done, conforme sugerido no enunciado da atividade.

Figura 1 – Visão geral do quadro Trello

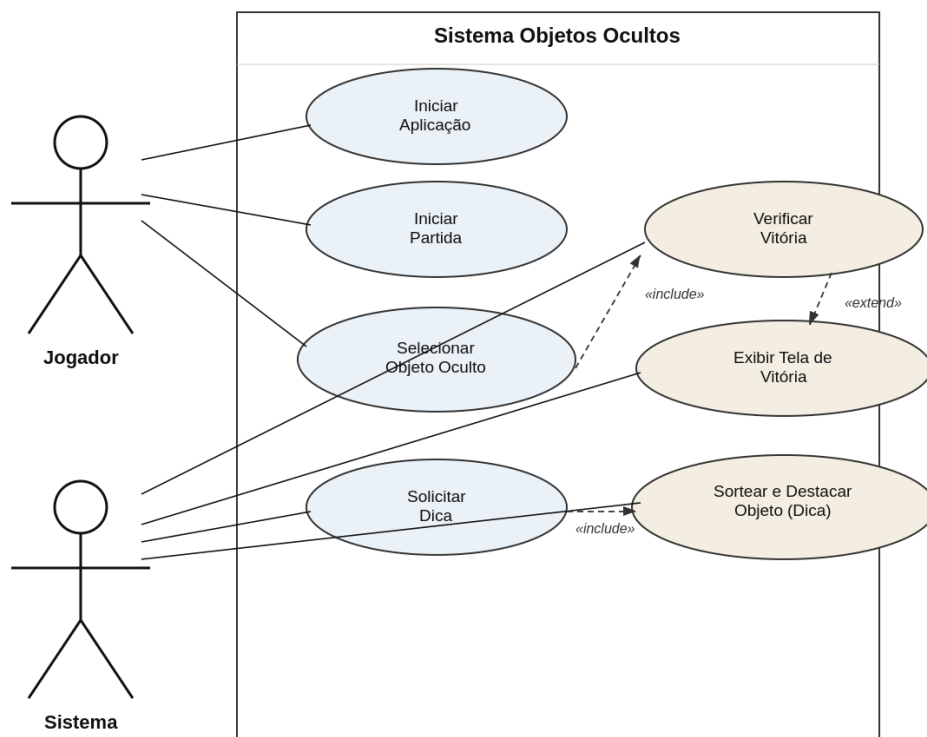


4 MODELAGEM UML

4.1 Diagrama de Casos de Uso

A partir da leitura do código do jogo, identificamos quatro casos de uso associados diretamente ao ator Jogador: iniciar a aplicação (abertura da tela de início), iniciar a partida (clique em “Jogar”, que abre a tela jogável), selecionar um objeto oculto (clique sobre qualquer item da cena) e solicitar uma dica (clique no botão “Dica”). Além do Jogador, identificamos também o ator Sistema, responsável pelas reações automáticas do jogo às ações do jogador: verificar vitória (conferir, a cada objeto encontrado, se a lista de itens restantes ficou vazia), exibir a tela de vitória (quando a verificação confirma que os nove itens foram encontrados) e sortear e destacar um objeto (ao receber uma solicitação de dica, escolher aleatoriamente um item ainda não encontrado e aplicar o efeito visual de destaque). O caso de uso “Selecionar Objeto Oculto”, do Jogador, inclui obrigatoriamente o caso de uso “Verificar Vitória”, do Sistema, já que essa verificação é chamada automaticamente a cada objeto encontrado; da mesma forma, “Solicitar Dica” inclui “Sortear e Destacar Objeto”. O caso de uso “Verificar Vitória” estende-se para “Exibir Tela de Vitória” apenas quando a lista de itens restantes está vazia.

Figura 2 – Diagrama de casos de uso do Objetos Ocultos



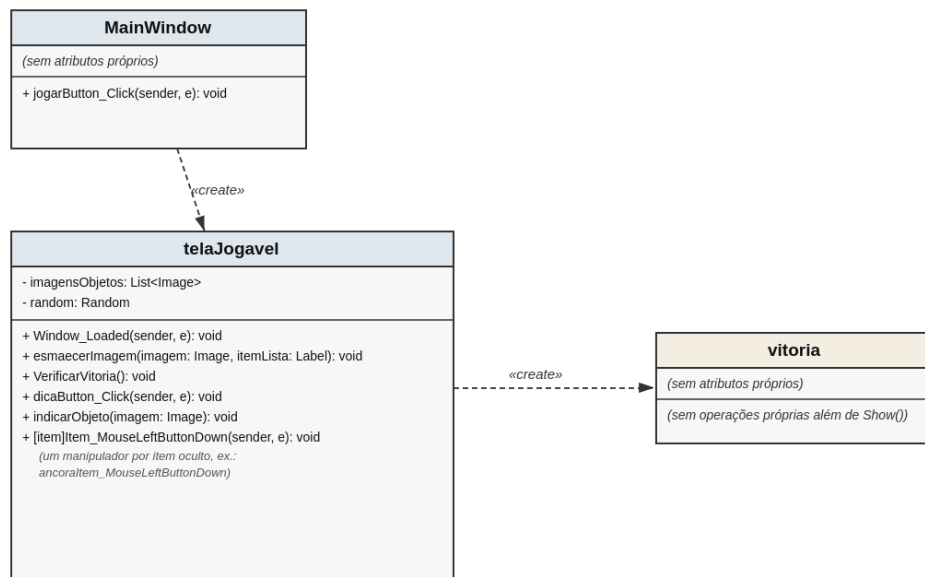
Fonte: Elaborado pelos autores (2026).

Fonte: Elaborado pelos autores

4.2 Diagrama de Classes

A partir da leitura do código-fonte, identificamos três classes principais que compõem a estrutura do Objetos Ocultos: `MainWindow`, responsável apenas pela tela de início e pelo clique no botão “Jogar”; `telaJogavel`, que concentra toda a lógica do jogo propriamente dito — a lista de itens (`imagensObjetos`), o sorteio de dicas, a animação de esmaecimento dos objetos e a verificação de vitória; e `vitoria`, uma janela simples exibida ao final da partida, sem lógica própria além de sua exibição. A classe `MainWindow` cria uma instância de `telaJogavel` ao iniciar a partida, e `telaJogavel` cria uma instância de `vitoria` quando os nove itens são encontrados, conforme representado pelas relações de dependência «create» no diagrama a seguir.

Figura 2.1 – Diagrama de classes do Objetos Ocultos



Fonte: Elaborado pelos autores (2026).

Fonte: Elaborado pelos autores

5 DESENVOLVIMENTO DO JOGO

Esta seção documenta, passo a passo, como o *Objetos Ocultos* foi construído. Como se trata de um jogo autoral e não de um jogo já existente, não há um “antes” de terceiros: cada tópico a seguir representa uma adição ao sistema, da ideia inicial até a verificação de vitória.

5.1 A Ideia

A proposta original da atividade era escolher um jogo pronto no catálogo “UUV Steam” e propor melhorias sobre ele. Depois de olhar as opções disponíveis, decidimos não seguir por esse caminho: preferimos construir um jogo nosso, do zero, mesmo sabendo que isso significava mais trabalho. Conversamos sobre alguns gêneros possíveis e chegamos a um jogo de objetos ocultos — um gênero clássico, de regra simples (encontrar todos os itens escondidos em uma cena para vencer), mas que exige atenção a detalhes de composição visual, opacidade e feedback ao jogador, o que se encaixava bem nos conceitos de GUI e orientação a objetos cobrados na atividade. Batizamos o jogo de *Objetos Ocultos*.

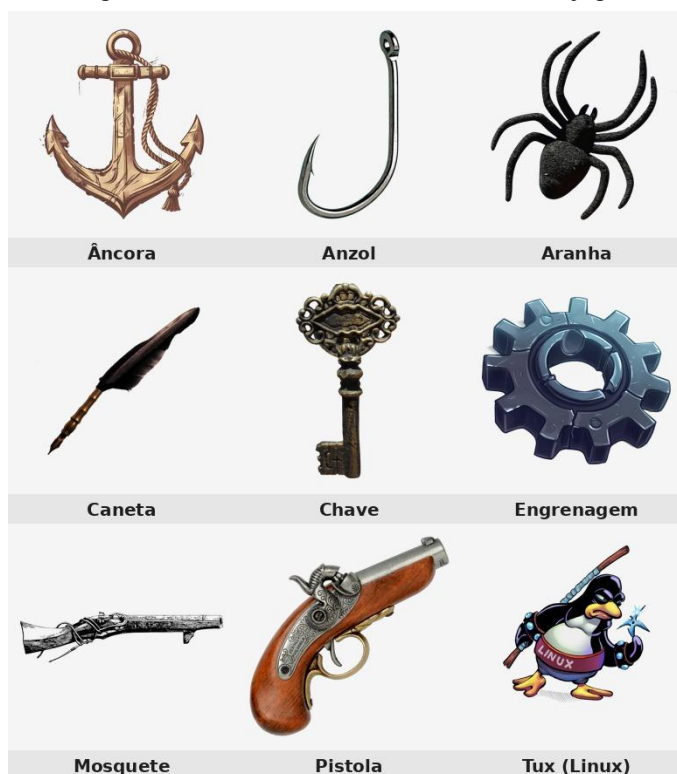
5.2 As Imagens

Todos os elementos visuais do jogo — cenário e itens ocultos — foram escolhidos e tratados manualmente pela dupla. Usamos uma imagem de fundo (*fundoJogoObjetos2.jpeg*) representando um cômodo antigo, com velas, potes, uma roca de fiar e um baú, para servir de cenário onde os itens seriam escondidos. Sobre esse fundo, posicionamos nove imagens de objetos em formato PNG com fundo transparente:

- Âncora (*ancora.png*) — ilustração de âncora em tons dourados/marrons, próximos à paleta do cenário de fundo;
- Anzol (*anzol.png*) — anzol metálico de silhueta fina, fácil de camuflar entre os objetos do quarto;
- Aranha (*aranha.png*) — silhueta preta de aranha, item clássico de jogos de “achar o objeto” por sua forma reconhecível;
- Caneta (*canetaPena.png*) — pena de escrever antiga, combinando com a ambientação do cômodo;
- Chave (*chave.png*) — chave ornamentada, em estilo antigo;

- Engrenagem (engrenagem.png) — engrenagem metálica, contrastando propositalmente com o restante do cenário;
- Mosquete (mosquete.png) — desenho de mosquete antigo, em traço fino;
- Pistola (pistola.png) — pistola antiga em tons de madeira e metal;
- Tux (tux.png) — o pinguim mascote do Linux, incluído como uma referência bem-humorada ao curso de Ciência da Computação.

Figura 3 – Os nove itens ocultos utilizados no jogo



Fonte: Elaborado pelos autores (2026).

5.3 Tela de Início

A primeira tela que o jogador vê é a MainWindow. Ela contém apenas o título do jogo e um botão “Jogar” sobre um fundo em gradiente azul-esverdeado. Ao clicar no botão, a tela jogável (telaJogavel) é aberta e exibida.

Figura 4 – Tela de início do Objetos Ocultos



Fonte: Elaborado pelos autores (2026).

Código 1 – Botão “Jogar” e título, em MainWindow.xaml

```
<Button x:Name="jogarButton" Content="JOGAR "
HorizontalAlignment="Center"
    VerticalAlignment="Center" Height="132" Width="262"
Click="jogarButton_Click" Foreground="White"
    FontFamily="Papyrus" FontSize="44">
    <Button.Background>
        <SolidColorBrush Color="#FF0A0544" Opacity="0.5"/>
    </Button.Background>
</Button>
<Label Content="Hidden Room" HorizontalAlignment="Center"
Margin="0,149,0,0"
    VerticalAlignment="Top" Height="143" Width="604"
FontFamily="Papyrus"
    FontSize="92" FontWeight="Bold" Foreground="#FF110B5A"/>
```

Fonte: MainWindow.xaml (2026).

Código 2 – Evento de clique do botão “Jogar”

```
private void jogarButton_Click(object sender, RoutedEventArgs e)
{
    telaJogavel abrirJogo = new telaJogavel();
    abrirJogo.Show();
}
```

Fonte: MainWindow.xaml.cs (2026).

5.4 Tela Jogável

A tela jogável (telaJogavel) tem 1600x900 pixels e usa a imagem de fundo como plano de fundo da janela (ImageBrush). Sobre um Grid, cada um dos nove itens é um controle Image posicionado manualmente com Margin e ajustado com um RenderTransform (ScaleTransform, SkewTransform, RotateTransform e TranslateTransform), para que cada objeto se encaixasse naturalmente na perspectiva e nos elementos do cenário. Na parte inferior, um retângulo

semitransparente sustenta os rótulos com o nome de cada item, e o botão “Dica” fica no canto direito.

Figura 5 – Tela jogável do Objetos Ocultos, com os itens escondidos no cenário



Fonte: Elaborado pelos autores (2026).

Código 3 – Posicionamento e transformação do item “Âncora”

```
<Image x:Name="ancoraItem" Margin="115,491,1414,286"
Source="/Images/ancora.png"
    Stretch="Fill" RenderTransformOrigin="0.5,0.5" Opacity="0.5"
    MouseLeftButtonDown="ancoraItem_MouseLeftButtonDown">
    <Image.RenderTransform>
        <TransformGroup>
            <ScaleTransform ScaleY="0.748" ScaleX="1.237"/>
            <SkewTransform AngleY="-8.55" AngleX="27.696"/>
            <RotateTransform Angle="26.106"/>
            <TranslateTransform Y="-77.503" X="24.295"/>
        </TransformGroup>
    </Image.RenderTransform>
</Image>
```

Fonte: telaJogavel.xaml (2026).

Código 4 – Registro dos itens na lista de controle do jogo

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    imagensObjetos.Add(aranhaItem);
    imagensObjetos.Add(ancoraItem);
    imagensObjetos.Add(anzolItem);
    imagensObjetos.Add(canetaItem);
    imagensObjetos.Add(chaveItem);
    imagensObjetos.Add(engrenagemItem);
    imagensObjetos.Add(mosqueteItem);
    imagensObjetos.Add(pistolaItem);
    imagensObjetos.Add(tuxItem);
}
```

Fonte: telaJogavel.xaml.cs (2026).

Essa lista (imagensObjetos) é a base de todo o controle de estado do jogo: é nela que o botão de dica e a verificação de vitória vão procurar quais itens ainda não foram encontrados, como mostram as próximas seções.

5.5 Imagens e suas Opacidades

Cada item recebeu uma opacidade inicial diferente, definida manualmente conforme o quão bem ele já se misturava ao cenário de fundo por conta de sua cor e forma. Itens com contraste natural baixo (como a aranha e o anzol) foram deixados totalmente opacos, enquanto itens muito destacados (como a pistola) precisaram de opacidade bem reduzida para não “pularem aos olhos” assim que a tela abre.

Quadro 1 – Opacidade inicial de cada item

Item	Opacidade	Observação
Âncora	0,50	Reduzida para se misturar aos tons de madeira do cenário.
Anzol	1,00 (padrão)	Silhueta fina, já pouco perceptível sobre o fundo.
Aranha	1,00 (padrão)	Contraste baixo por já ser um formato reconhecível em meio à decoração.
Caneta	0,748	Levemente reduzida para não competir com os objetos próximos.
Chave	0,95	Quase opaca; sua textura antiga já camufla bem no cenário.
Engrenagem	0,80	Reduzida por ser um objeto com bastante contraste (metálico) sobre o fundo.
Mosquete	0,95	Traço fino, exige pouca redução de opacidade.
Pistola	0,35	O item mais destacado da cena; precisou da maior redução para ficar bem escondido.
Tux	0,40	Cores vivas (preto/branco/laranja) exigiram bastante redução para se camuflar.

Fonte: Elaborado pelos autores (2026).

5.6 Animação e Função de Esmaecer

Quando o jogador clica em um item, o método `esmaecerImagem` é chamado. Ele cria uma `DoubleAnimation` que reduz a opacidade da imagem de 1.0 para 0.0 em um segundo, fazendo o objeto desaparecer suavemente da cena. Ao término da animação (evento `Completed`), o método `VerificarVitoria` é chamado, checando se ainda restam itens por encontrar. Ao mesmo tempo, o rótulo correspondente na lista inferior tem sua opacidade reduzida para 0.4, indicando visualmente que aquele item já foi encontrado.

Código 5 – Função esmaecerImagem

```
private void esmaecerImagem(Image imagem, Label itemLista)
{
    DoubleAnimation animacao = new DoubleAnimation()
    {
        From = 1.0,
        To = 0.0,
        Duration = TimeSpan.FromSeconds(1)
    };

    animacao.Completed += (s, e) =>
    {
        VerificarVitoria();
    };

    imagem.BeginAnimation(UIElement.OpacityProperty, animacao);

    itemLista.Opacity = 0.4;
}
```

Fonte: telaJogavel.xaml.cs (2026).

Cada um dos nove itens tem seu próprio manipulador de clique (por exemplo, `ancoraItem_MouseLeftButtonDown`), mas todos eles apenas repassam a chamada para `esmaecerImagem`, passando a imagem e o rótulo correspondentes.

5.7 Botão de Dica

O botão “Dica” chama `dicaButton_Click`, que filtra a lista `imagensObjetos` mantendo apenas os itens ainda com opacidade maior que zero (ou seja, ainda não encontrados), sorteia um deles com `Random` e chama `indicarObjeto`. Essa função aplica um efeito de brilho branco (`DropShadowEffect`) ao item sorteado e o faz pulsar: a opacidade do brilho e a escala da imagem oscilam cinco vezes entre os valores definidos, chamando a atenção do jogador sem revelar o item de forma definitiva — ele continua escondido, apenas destacado por alguns instantes.

Código 6 – Evento de clique do botão “Dica”

```
private void dicaButton_Click(object sender, RoutedEventArgs e)
{
    List<Image> objetosDisponiveis = imagensObjetos
        .Where(Imagem => Imagem.Opacity > 0)
        .ToList();

    if (objetosDisponiveis.Count == 0)
    {
        return;
    }

    int indice = random.Next(objetosDisponiveis.Count);
    Image objetoEscolhido = objetosDisponiveis[indice];
}
```

```

    indicarObjeto(objetoEscolhido);
}

```

Fonte: telaJogavel.xaml.cs (2026).

Código 7 – Função indicarObjeto (destaque pulsante do item sorteado)

```

private void indicarObjeto(Image imagem)
{
    DropShadowEffect efeito = new DropShadowEffect()
    {
        Color = Colors.White,
        BlurRadius = 50,
        ShadowDepth = 0,
        Opacity = 0
    };

    imagem.Effect = efeito;

    DoubleAnimation animacaoBrilho = new DoubleAnimation()
    {
        From = 0.0,
        To = 1.0,
        Duration = TimeSpan.FromSeconds(0.4),
        AutoReverse = true,
        RepeatBehavior = new RepeatBehavior(5)
    };

    ScaleTransform escala = new ScaleTransform(1, 1);
    imagem.RenderTransform = escala;
    imagem.RenderTransformOrigin = new Point(0.5, 0.5);

    DoubleAnimation animacaoEscala = new DoubleAnimation()
    {
        From = 1.0,
        To = 1.35,
        Duration = TimeSpan.FromSeconds(0.4),
        AutoReverse = true,
        RepeatBehavior = new RepeatBehavior(5)
    };

    efeito.BeginAnimation(DropShadowEffect.OpacityProperty,
        animacaoBrilho);
    escala.BeginAnimation(ScaleTransform.ScaleXProperty,
        animacaoEscala);
    escala.BeginAnimation(ScaleTransform.ScaleYProperty,
        animacaoEscala);
}

```

Fonte: telaJogavel.xaml.cs (2026).

5.8 Vitória Verificada na Função Esmaecer

A verificação de vitória não tem um gatilho próprio: ela é chamada de dentro do callback Completed da animação de esmaecerImagem, logo após cada item desaparecer. O método

VerificarVitoria filtra a lista imagensObjetos pelos itens que ainda têm opacidade maior que zero; se essa lista estiver vazia — ou seja, se os nove itens já tiverem sido encontrados — uma nova janela vitoria é criada e exibida.

Código 8 – Função VerificarVitoria

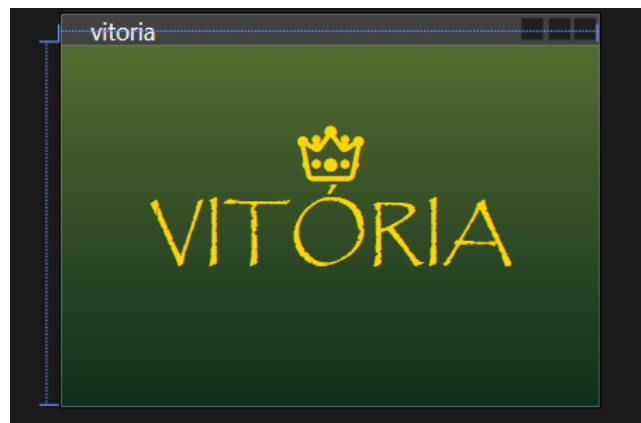
```
private void VerificarVitoria()
{
    List<Image> objetosDisponiveis = imagensObjetos
        .Where(Imagem => Imagem.Opacity > 0)
        .ToList();

    if (objetosDisponiveis.Count == 0)
    {
        vitoria ganhou = new vitoria();
        ganhou.Show();
    }
}
```

Fonte: telaJogavel.xaml.cs (2026).

A janela vitoria é bem simples: um fundo em gradiente (verde-escuro para verde-oliva), o texto “VITÓRIA” em dourado com a fonte Papyrus — a mesma usada nos rótulos do jogo, para manter a identidade visual — e um emoji de coroa acima do texto.

Figura 6 – Tela de vitória, exibida quando os nove itens são encontrados



Fonte: Elaborado pelos autores (2026).

Código 9 – Composição visual da tela de vitória

```
<Grid.Background>
    <LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0">
        <GradientStop Color="#FF0E2F1D" Offset="1"/>
        <GradientStop Color="DarkOliveGreen" Offset="0"/>
    </LinearGradientBrush>
</Grid.Background>
<Label Content="VITÓRIA" HorizontalAlignment="Center"
    VerticalAlignment="Top"
        FontFamily="Papyrus" FontSize="36" Margin="0,64,0,0"
    Foreground="Gold"/>
```



```
<Label Content="👑" HorizontalAlignment="Center" Margin="0,24,0,0"  
        VerticalAlignment="Top" FontSize="32" Foreground="Gold"/>
```

Fonte: vitoria.xaml (2026).

6 CONSIDERAÇÕES FINAIS

Optar por construir um jogo autoral em vez de contribuir com um projeto pronto do catálogo trouxe um desafio diferente do previsto originalmente pela atividade: em vez de entender a lógica de outra pessoa, tivemos que tomar todas as decisões de design, arte e programação do zero. A parte que mais exigiu ajustes finos foi justamente o equilíbrio das opacidades dos itens — um objeto opaco demais fica óbvio à primeira vista, e um objeto de menos opacidade demais se torna praticamente impossível de encontrar, então cada valor do Quadro 1 foi testado visualmente até parecer justo.

REFERÊNCIAS

MICROSOFT. Windows Presentation Foundation documentation. Redmond: Microsoft, [2026]. Disponível em: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>. Acesso em: 3 set. 2026.

UNIVERSIDADE VILA VELHA. Atividade Avaliativa: UML, GUI, Jogos. Vila Velha: UVV, 2026.